

RoboScene to CSP Translation Rules

Holly Hendry

April 20, 2026

Rule 1. RoboScene model
$$\llbracket \mathbf{rsm : RoboSceneModel} \rrbracket_M \hat{=}$$
$$\llbracket \mathbf{rsm.constant} \rrbracket_{CO} \llbracket \mathbf{rsm.capabilities} \rrbracket_{ABS} \quad (1)$$
$$\llbracket \mathbf{rsm.sequencediagramgroup} \rrbracket_{SDGS} \llbracket \mathbf{rsm.types} \rrbracket_{TY} \quad (2)$$

Rule 2. Constants
$$\llbracket \mathbf{varList : VariableList, const : Constants, timeunit : TimeUnit} \rrbracket_{CO} \hat{=}$$
$$\llbracket \mathbf{varList} \rrbracket_{CON} \text{ (if } \mathbf{const} \neq \mathbf{null} \text{ then } \llbracket \mathbf{const, timeunit} \rrbracket_{CO} \text{)} \quad (1)$$

Rule 3. Constants List
$$\llbracket \mathbf{const : Variable, constList : VariableList} \rrbracket_{CON} \hat{=}$$
$$\llbracket \mathbf{const} \rrbracket_C \text{ (if } \mathbf{constList} \neq \mathbf{null} \text{ then } \llbracket \mathbf{const.constantsconstList} \rrbracket_{CON} \text{)} \quad (1)$$

Rule 4. Constant Specification
$$\llbracket (\mathbf{const\ constID = val}) : \mathbf{Variable} \rrbracket_C \hat{=}$$
$$\mathbf{constID = val} \quad (1)$$

Rule 5. Capabilities
$$\llbracket \mathbf{ab : ActorBlock, abs : seq ActorBlock} \rrbracket_{ABS} \hat{=}$$
$$\llbracket \mathbf{ab} \rrbracket_{AB} \text{ (if } \mathbf{abs} \neq \mathbf{null} \text{ then } \llbracket \mathbf{abs} \rrbracket_{ABS} \text{)} \quad (1)$$

Rule 6. ActorBlock
$$\llbracket \mathbf{ab : ActorBlock} \rrbracket_{AB} \hat{=}$$
$$\llbracket \mathbf{ab.outs} \rrbracket_{LES} \llbracket \mathbf{ab.ins} \rrbracket_{LES} \llbracket \mathbf{ab.variables} \rrbracket_{VARS} \quad (1)$$

Rule 7. Lifeline Events
$$\llbracket \mathbf{event : LifelineEvents, les : seq LifelineEvents} \rrbracket_{LES} \hat{=}$$
$$\llbracket \mathbf{varList} \rrbracket_{VAR} \text{ (if } \mathbf{varLists} \neq \mathbf{null} \text{ then } \llbracket \mathbf{varLists} \rrbracket_{VARS} \text{)} \quad (1)$$

Rule 8. Lifeline Out Event

$$\llbracket (\text{out event} \rightarrow \text{actor}) : \mathbf{LifelineEvents} \rrbracket_{LE} \hat{=}$$

$$\text{channel event} \quad (1)$$

Rule 9. Typed Lifeline Out Event

$$\llbracket (\text{out event} : \text{Type} \rightarrow \text{actor}) : \mathbf{LifelineEvents} \rrbracket_{LE} \hat{=}$$

$$\text{channel event} : \llbracket \text{Type} \rrbracket_{TYPEEXP} \quad (1)$$

Rule 10. Lifeline In Event

$$\llbracket (\text{in event} \leftarrow \text{actor}) : \mathbf{LifelineEvents} \rrbracket_{LE} \hat{=}$$

$$\text{channel event} \quad (1)$$

Rule 11. Typed Lifeline In Event

$$\llbracket (\text{in event} : \text{Type} \leftarrow \text{actor}) : \mathbf{LifelineEvents} \rrbracket_{LE} \hat{=}$$

$$\text{channel event} : \llbracket \text{Type} \rrbracket_{TYPEEXP} \quad (1)$$

Rule 12. Variable List

$$\llbracket \text{varList} : \mathbf{VariableList}, \text{varLists} : \text{seq } \mathbf{VariableList} \rrbracket_{VARS} \hat{=}$$

$$\llbracket \text{varList} \rrbracket_{VAR} \text{ (if varLists } \neq \text{ null then } \llbracket \text{varLists} \rrbracket_{VARS} \text{)} \quad (1)$$

Rule 13. Variable List

$$\llbracket \text{var} : \mathbf{Variable}, \text{varList} : \mathbf{VariableList} \rrbracket_{VAR} \hat{=}$$

$$\llbracket \text{var} \rrbracket_V \text{ (if varList } \neq \text{ null then } \llbracket \text{varList} \rrbracket_{VAR} \text{)} \quad (1)$$

Rule 14. Variable Specification

$$\llbracket (\text{var} : \text{Type}) : \mathbf{Variable} \rrbracket_V \hat{=}$$

$$\text{channel set_var} : \llbracket \text{Type} \rrbracket_{TYPEEXP} \quad (1)$$

$$\text{channel get_var} : \llbracket \text{Type} \rrbracket_{TYPEEXP} \quad (2)$$

Rule 15. Sequence Diagram Groups

$$\llbracket \text{sdg} : \text{SequenceDiagramGroup}, \text{sdgs} : \text{seq SequenceDiagramGroup} \rrbracket_{SDGS} =$$

$$\llbracket \text{sdg} \rrbracket_{SDG} \text{ (if } \text{sdgs} \neq \text{null then } \llbracket \text{sdqs} \rrbracket_{SDGS} \text{)}$$

(1)

Rule 16. Sequence Diagram Group

$$\llbracket \text{s} : \text{Sequence}, \text{sdg} : \text{SequenceDiagramGroup} \rrbracket_{SDG} =$$

$$\llbracket \text{s} \rrbracket_S \text{ (if } \text{sdg} \neq \text{null then } \llbracket \text{sdq} \rrbracket_{SDG} \text{)}$$

(1)

Rule 17. Sequence

$$\llbracket \text{s} : \text{Sequence} \rrbracket_S \hat{=}$$

$$(((\llbracket \text{a} : \text{actors} \bullet \llbracket \text{a}; \text{s.fragment} \rrbracket_A \llbracket \text{a}; \text{s.fragment} \rrbracket_{LIF} \rrbracket_{(1)} \\ \llbracket \{ \text{str}, \text{par}, \text{terminate} \} \rrbracket \llbracket \text{parFragments}; \text{strFragments} \rrbracket_{CTRL} \rrbracket_{(2)} \\ \llbracket \{ \text{alt}, \text{opt}, \text{loop}, \text{guard}, \text{terminate} \} \rrbracket \llbracket \text{altFragments}; \text{loopFragments}; \text{optFragments} \rrbracket_{GRD} \rrbracket_{(3)} \\ \llbracket \llbracket \text{s.variable} \rrbracket_{SVARS} \cup \{ \text{terminate} \} \rrbracket \llbracket \text{s.variable} \rrbracket_{MEMV} \rrbracket_{(4)} \\ \setminus \{ \text{alt}, \text{opt}, \text{loop}, \text{guard}, \text{str}, \text{par} \} \rrbracket_{(5)} \\ \text{datatype IDs} = ID_ALT \mid ID_OPT \mid ID_LOOP \mid ID_PAR \mid ID_STR \rrbracket_{(6)} \\ \text{where} \\ \text{actors} = \text{s.actor} \text{ and } \text{parFragments} = \text{par}(\text{s.fragment}) \text{ and } \rrbracket_{(7)} \\ \text{strFragments} = \text{str}(\text{s.fragment}) \text{ and } \text{altFragments} = \text{alt}(\text{s.fragment}) \text{ and } \rrbracket_{(8)} \\ \text{optFragments} = \text{opt}(\text{s.fragment}) \text{ and } \text{loopFragments} = \text{loop}(\text{s.fragment}) \rrbracket_{(9)}$$

Rule 18. Actor Alphabet

$$\llbracket \text{a} : \text{Actor}, \text{frags} : \text{seq InteractionFragment} \rrbracket_A \hat{=}$$

$$\{ \llbracket \text{actorCaps}(\text{a}) \rrbracket \rrbracket_{(1)} \\ \cup \{ \llbracket \text{f} : \text{actFragmentsL}(\text{frags}, \text{a}) \bullet \text{loop.ID_LOOP.id}(\text{f}), \text{guard.ID_LOOP.id}(\text{f}) \rrbracket \rrbracket_{(2)} \\ \cup \{ \llbracket \text{f} : \text{actFragmentsA}(\text{frags}, \text{a}) \bullet \text{alt.ID_ALT.id}(\text{f}), \text{guard.ID_ALT.id}(\text{f}) \rrbracket \rrbracket_{(3)} \\ \cup \{ \llbracket \text{f} : \text{actFragmentsO}(\text{frags}, \text{a}) \bullet \text{opt.ID_OPT.id}(\text{f}), \text{guard.ID_OPT.id}(\text{f}) \rrbracket \rrbracket_{(4)} \\ \cup \{ \llbracket \text{f} : \text{actFragmentsP}(\text{frags}, \text{a}) \bullet \text{par.ID_PAR.id}(\text{f}) \rrbracket \rrbracket_{(5)} \\ \cup \{ \llbracket \text{f} : \text{actFragmentsS}(\text{frags}, \text{a}) \bullet \text{str.ID_STR.id}(\text{f}) \rrbracket \rrbracket_{(6)} \\ \cup \{ \llbracket \text{terminate} \rrbracket \rrbracket_{(7)}$$

Rule 19. Actor Lifeline

$$\llbracket \mathbf{a} : \mathbf{Actor}, \mathbf{frags} : \text{seq InteractionFragment} \rrbracket_{LIF} \hat{=} \\ \text{;} \text{ frag} : \text{actFrag}(\text{frags}) \bullet \llbracket \text{frag} \rrbracket_{(\mathbf{a})}^F \quad (1)$$

Rule 20. Actor Control Fragments

$$\llbracket \mathbf{parFrag} : \mathbf{ParFragment} \rrbracket_{(a)}^F = \\ \text{par.ID_PAR.id}(\text{parFrag}) \rightarrow \text{Skip} \quad (1)$$

Rule 21. Actor Control Fragments

$$\llbracket \mathbf{strFrag} : \mathbf{StrFragment} \rrbracket_{(a)}^F = \\ \text{str.ID_STR.id}(\text{strFrag}) \rightarrow \text{Skip} \quad (1)$$

Rule 22. Actor Guarded Fragments

$$\llbracket \mathbf{altFrag} : \mathbf{AltFragment} \rrbracket_{(\mathbf{a})}^F = \\ \llbracket \text{altFrag}, \mathbf{a} \rrbracket_{AF} \quad (1)$$

Rule 23. Actor Guarded Fragments

$$\llbracket \mathbf{altFrag} : \mathbf{AltFragment} \rrbracket_{(\mathbf{a})}^F = \\ \llbracket \text{optFrag}, \mathbf{a} \rrbracket_{OF} \quad (1)$$

Rule 24. Actor Guarded Fragments

$$\llbracket \mathbf{altFrag} : \mathbf{AltFragment} \rrbracket_{(\mathbf{a})}^F = \\ \llbracket \text{loopFrag}, \mathbf{a} \rrbracket_{LF} \quad (1)$$

Rule 25. Control

$$\llbracket \mathbf{parFrag}, \mathbf{strFrag} \rrbracket_{CTRL} \hat{=} \\ \llbracket \text{parFrag} \rrbracket_{PAR} \parallel \{ \text{terminate} \} \parallel \llbracket \text{strFrag} \rrbracket_{STR} \quad (1)$$

$$\text{channel par} : \text{IDs.Int} \quad (2)$$

$$\text{channel str} : \text{IDs.Int} \quad (3)$$

Rule 26. Parallel
$$\llbracket \mathbf{parFrag} \rrbracket_{PAR} \hat{=}$$

let
$$Parallel = \quad (1)$$
$$par.ID_PAR?id \rightarrow \quad (2)$$
$$(\square \text{ parFrag} : \text{parFrag} \bullet \quad (3)$$
$$(id == \underline{\text{id}(\text{parFrag})}) \& \quad (4)$$
$$(\parallel \text{ branch} : \text{parFrag.branches} \bullet \llbracket \text{parFrag}, \text{branch} \rrbracket_{PT}) \quad (5)$$
$$) \circ \text{ Parallel} \quad (6)$$
$$\square \text{ terminate} \rightarrow \text{Skip} \quad (7)$$
within
$$Parallel \quad (8)$$

Rule 27. Parallel Thread
$$\llbracket \mathbf{parFrag}, \mathbf{branch} \rrbracket_{PT} \hat{=}$$
$$\parallel \text{ actor} : \text{branch.actors} \bullet \quad (1)$$
$$\llbracket \text{actor}, \text{parFrag.branch.fragments} \rrbracket_A \llbracket \text{actor}, \text{parFrag}, \text{branch} \rrbracket_{PTA} \quad (2)$$

Rule 28. Parallel Thread per Actor
$$\llbracket \mathbf{actor}, \mathbf{parFrag}, \mathbf{branch} \rrbracket_{PTA} \hat{=}$$
$$\llbracket \text{actor}, \text{parFrag.branch.fragments} \rrbracket_{LIF} \quad (1)$$

Rule 29. Strict
$$\llbracket \mathbf{strFrag} \rrbracket_{STR} \hat{=}$$

let
$$Strict = \quad (1)$$
$$str.ID_STR?id \rightarrow \quad (2)$$
$$(\square \text{ strFrag} : \text{strFrag} \bullet \quad (3)$$
$$(id == \underline{\text{id}(\text{strFrag})}) \& \llbracket \text{strFrag} \rrbracket_{SF}) \quad (4)$$
$$) \circ \text{ Strict} \quad (5)$$
$$\square \text{ terminate} \rightarrow \text{Skip} \quad (6)$$
within
$$Strict \quad (7)$$

Rule 30. Strict Fragment
$$\llbracket \mathbf{strFrag} \rrbracket_{SF} \hat{=}$$
$$\circ \text{ frag} : \text{strFrag.fragments} \bullet \llbracket \text{frag} \rrbracket_{(a)}^F \quad (1)$$

Rule 31. Guard
$$\llbracket \text{altFrag}s, \text{loopFrag}s, \text{optFrag}s \rrbracket_{GRD} \hat{=}$$

$$(\llbracket \text{altFrag}s, \text{loopFrag}s, \text{optFrag}s \rrbracket_{EVAL} \quad (1)$$

$$|| \text{alphaCounters} \cup \{| \text{terminate} |\} || \llbracket \text{loopFrag}s \rrbracket_{CNT} \setminus \text{alphaCounters} \quad (2)$$

$$\text{channel alt} : \text{IDs.Int} \quad (3)$$

$$\text{channel opt} : \text{IDs.Int} \quad (4)$$

$$\text{channel loop} : \text{IDs.Int} \quad (5)$$

$$\text{channel guard} : \text{IDs.Int.Int.Bool} \quad (6)$$

$$\text{channel getCount} : \text{IDs.Int.Int} \quad (7)$$

$$\text{channel setCount} : \text{IDs.Int.Int} \quad (8)$$

$$\text{alphaCounters} = \{| \text{getCount}, \text{setCount} |\} \quad (9)$$

Rule 32. Count
$$\llbracket \text{loopFrag}s \rrbracket_{CNT} \hat{=}$$

$$(| | \text{loopID} : \text{loopIDs} \bullet \{| \text{terminate} |\} | \llbracket \text{loopID} \rrbracket_{CNTID}) \quad (1)$$

where

$$\text{loopIDs} = \text{ids}(\text{loopFrag}s) \quad (2)$$

Rule 33. Count per Fragment
$$\llbracket \text{loopID} \rrbracket_{CNTID} \hat{=}$$

let

$$\text{Counter}(\text{count}) = \quad (1)$$

$$\text{getCount.ID_LOOP.loopID!count} \rightarrow \text{Counter}(\text{count}) \quad (2)$$

$$\square \text{ setCount.ID_LOOP.loopID?y} \rightarrow \text{Counter}(y) \quad (3)$$

$$\square \text{ terminate} \rightarrow \text{Skip} \quad (4)$$

within

$$\text{Counter}(0) \quad (5)$$

Rule 34. Evaluation
$$\llbracket \text{altFrag}s, \text{loopFrag}s, \text{optFrag}s \rrbracket_{EVAL} \hat{=}$$

$$\llbracket \text{loopFrag}s \rrbracket_{RES} \circ \llbracket \text{altFrag}s, \text{loopFrag}s, \text{optFrag}s \rrbracket_{GRSP} \quad (1)$$

Rule 35. Reset Counters
$$\llbracket \text{loopFrag} \rrbracket_{RES} \hat{=}$$

$$\text{loopID} : \text{loopIDs} \bullet \text{setCount.ID_LOOP.loopID!0} \rightarrow \text{Skip} \quad (1)$$

where

$$\text{loopIDs} = \text{ids}(\text{loopFrag}) \quad (2)$$

Rule 36. Guard Response
$$\llbracket \text{altFrag}, \text{loopFrag}, \text{optFrag} \rrbracket_{GRSP} \hat{=}$$

let

$$\text{GuardResponse} = \quad (1)$$

$$\square \text{alt?ID_ALT.id} \rightarrow (\quad (2)$$

$$\square \text{altFrag} : \text{altFrag} \bullet (id == \text{id}(\text{altFrag})) \& (\llbracket \text{altFrag} \rrbracket_{AEV}) \quad (3)$$

$$) \text{ } \text{GuardResponse} \quad (4)$$

$$\square \text{opt?ID_OPT.id} \rightarrow (\quad (5)$$

$$\square \text{optFrag} : \text{optFrag} \bullet (id == \text{id}(\text{optFrag})) \& (\llbracket \text{optFrag} \rrbracket_{OEV}) \quad (6)$$

$$) \text{ } \text{GuardResponse} \quad (7)$$

$$\square \text{loop?ID_LOOP.id} \rightarrow (\quad (8)$$

$$\square \text{loopFrag} : \text{loopFrag} \bullet (id == \text{id}(\text{loopFrag})) \& (\llbracket \text{loopFrag} \rrbracket_{LEV}) \quad (9)$$

$$) \text{ } \text{GuardResponse} \quad (10)$$

$$\square \text{terminate} \rightarrow \text{Skip} \quad (11)$$

within

$$\text{GuardResponse} \quad (12)$$

Rule 37. Alt Fragment Evaluation
$$\llbracket \text{altFrag} \rrbracket_{AEV} \hat{=}$$

$$(\rightarrow \text{variable} : \text{guardVariables} \bullet (\text{get_variable})) \quad (1)$$

$$\rightarrow (\llbracket \text{altFrag} \rrbracket_G) \quad (2)$$

where

$$\text{guardVariables} = \text{getFragmentGuardVariables}(\text{altFrag}) \quad (3)$$

Rule 38. Opt Fragment Evaluation $\mathbf{optFrag}_{OEV} \hat{=}$

$$(\rightarrow \text{variable} : \text{guardVariables} \bullet (\underline{\text{get_variable}})) \quad (1)$$

$$\rightarrow ([\text{optFrag}]_G) \quad (2)$$

where

$$\text{guardVariables} = \underline{\text{getFragmentGuardVariables}(\text{optFrag})} \quad (3)$$

Rule 39. Loop Fragment Evaluation $\mathbf{loopFrag}_{LEV} \hat{=}$

$$(\rightarrow \text{variable} : \text{guardVariables} \bullet (\underline{\text{get_variable}})) \quad (1)$$

$$\rightarrow ([\text{loopFrag}]_G) \quad (2)$$

where

$$\text{guardVariables} = \underline{\text{getFragmentGuardVariables}(\text{loopFrag})} \quad (3)$$

Rule 40. Alt Fragment Guard
$$\frac{[(\text{alt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] \text{ } x_1 \text{ else } \dots \text{ else } [g_{-n_0} \& g_{-n_1} \& \dots \& g_{-n_n}] \text{ } x_n \text{ else } x_m \text{ end}) : \mathbf{AltFragment}]]_G}{=}$$

$$(g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \& (\text{guard.ID_ALT.id}(\underline{\text{altFrag}}).1!\text{true} \rightarrow \dots) \quad (1)$$

$$\rightarrow \text{guard.ID_ALT.id}(\underline{\text{altFrag}}).n!\text{false}) \quad (2)$$

$$\square \quad (3)$$

$$\dots \quad (4)$$

$$\square \quad (5)$$

$$(g_{-n_0} \& g_{-n_1} \& \dots \& g_{-n_n}) \& (\text{guard.ID_ALT.id}(\underline{\text{altFrag}}).1!\text{false} \rightarrow \dots) \quad (6)$$

$$\rightarrow \text{guard.ID_ALT.id}(\underline{\text{altFrag}}).n!\text{true}) \quad (7)$$

$$\square \quad (8)$$

$$\text{not}((g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \text{ or } \dots \text{ or } (g_{-n_0} \& g_{-n_1} \& \dots \& g_{-n_n})) \& \quad (9)$$

$$(\text{guard.ID_ALT.id}(\underline{\text{altFrag}}).1!\text{false} \rightarrow \dots \rightarrow \quad (10)$$

$$\text{guard.ID_ALT.id}(\underline{\text{altFrag}}).n!\text{false}) \quad (11)$$

Rule 41. Alt Fragment Guard
$$\llbracket (\text{alt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] \ x_1 \ \text{else } \dots \ \text{else } [g_{-n_0} \& g_{-n_1} \& \dots \& g_{-n_n}] \ x_n \ \text{end}) : \mathbf{AltFragment} \rrbracket_G =$$

$$(g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \& (\text{guard.ID_ALT.id}(\text{altFrag}).1!true \rightarrow \dots) \quad (1)$$

$$\rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).n!false) \quad (2)$$

$$\square \quad (3)$$

$$\dots \quad (4)$$

$$\square \quad (5)$$

$$(g_{-n_0} \& g_{-n_1} \& \dots \& g_{-n_n}) \& (\text{guard.ID_ALT.id}(\text{altFrag}).1!false \rightarrow \dots) \quad (6)$$

$$\rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).n!true) \quad (7)$$

$$\square \quad (8)$$

$$\text{not}((g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \ \text{or } \dots \ \text{or } (g_{-n_0} \& g_{-n_1} \& \dots \& g_{-n_n})) \& \quad (9)$$

$$(\text{guard.ID_ALT.id}(\text{altFrag}).1!false \rightarrow \dots \rightarrow \quad (10)$$

$$\text{guard.ID_ALT.id}(\text{altFrag}).n!false) \quad (11)$$

Rule 42. Alt Fragment Guard
$$\llbracket (\text{alt } x_1 \ \text{else } \dots \ \text{else } x_n \ \text{end}) : \mathbf{AltFragment} \rrbracket_G =$$

$$(\text{guard.ID_ALT.id}(\text{altFrag}).1!true \rightarrow \dots) \quad (1)$$

$$\rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).n!false) \quad (2)$$

$$\square \quad (3)$$

$$\dots \quad (4)$$

$$\square \quad (5)$$

$$(\text{guard.ID_ALT.id}(\text{altFrag}).1!false \rightarrow \dots) \quad (6)$$

$$\rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).n!true) \quad (7)[2pt]$$

Rule 43. Alt Fragment Guard
$$\llbracket (\text{alt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] \ x_1 \ \text{else } x_2 \ \text{end}) : \mathbf{AltFragment} \rrbracket_G =$$

$$(g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \& (\text{guard.ID_ALT.id}(\text{altFrag}).1!true) \quad (1)$$

$$\square \quad (2)$$

$$\text{not}(g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \& (\text{guard.ID_ALT.id}(\text{altFrag}).1!false) \quad (4)$$

Rule 44. Alt Fragment Guard
$$\llbracket (\text{alt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] \ x_1 \ \text{end}) : \mathbf{AltFragment} \rrbracket_G =$$

$$(g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \& (\text{guard.ID_ALT.id}(\text{altFrag}).1!true) \quad (1)$$

$$\square \quad (2)$$

$$\text{not}(g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \& (\text{guard.ID_ALT.id}(\text{altFrag}).1!false) \quad (4)$$

Rule 45. Opt Fragment Guard
$$\llbracket (\text{opt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] \ x_1 \ \text{end}) : \mathbf{OptFragment} \rrbracket_G =$$

$$(g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \& (\text{guard.ID_OPT.id}(\text{optFrag}).1!true) \quad (1)$$

$$\square \quad (2)$$

$$\text{not}(g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}) \& (\text{guard.ID_OPT.id}(\text{optFrag}).1!false) \quad (4)$$

Rule 46. Loop Fragment Guard
$$\llbracket (\text{loop}(\text{min}, \text{max})[\text{condition}] \ x_1 \ \text{end}) : \mathbf{LoopFragment} \rrbracket_G =$$

$$\text{not}(\text{condition}) \& \quad (1)$$

$$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!0 \rightarrow \quad (2)$$

$$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!false \rightarrow \text{Skip}) \quad (3)$$

$$\square \quad (4)$$

$$((\text{count} < \text{min}) \text{ and } \text{condition}) \& \quad (5)$$

$$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!(\text{count} + 1) \rightarrow \quad (6)$$

$$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!true \rightarrow \text{Skip}) \quad (7)$$

$$\square \quad (8)$$

$$(\text{count} == \text{max}) \& \quad (9)$$

$$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!0 \rightarrow \quad (10)$$

$$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!false \rightarrow \text{Skip}) \quad (11)$$

$$\square \quad (12)$$

$$((\text{count} \geq \text{min}) \text{ and } (\text{count} < \text{max}) \text{ and } \text{condition}) \& \quad (13)$$

$$((\text{setCount.ID_LOOP.id}(\text{loopFrag})!(\text{count} + 1) \rightarrow \quad (14)$$

$$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!true \rightarrow \text{Skip}) \sqcap \quad (15)$$

$$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!0 \rightarrow \quad (16)$$

$$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!false \rightarrow \text{Skip})) \quad (17)$$

Rule 47. Loop Fragment Guard
$$\llbracket (\text{loop}(\text{min}, \text{max}) \ x_1 \ \text{end}) : \mathbf{LoopFragment} \rrbracket_G =$$

$$(\text{count} < \text{min}) \& \quad (1)$$

$$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!(\text{count} + 1) \rightarrow \quad (2)$$

$$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!true \rightarrow \text{Skip}) \quad (3)$$

$$\square \quad (4)$$

$$(\text{count} == \text{max}) \& \quad (5)$$

$$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!0 \rightarrow \quad (6)$$

$$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!false \rightarrow \text{Skip}) \quad (7)$$

$$\square \quad (8)$$

$$((\text{count} \geq \text{min}) \text{ and } (\text{count} < \text{max})) \& \quad (9)$$

$$((\text{setCount.ID_LOOP.id}(\text{loopFrag})!(\text{count} + 1) \rightarrow \quad (10)$$

$$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!true \rightarrow \text{Skip}) \sqcap \quad (11)$$

$$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!0 \rightarrow \quad (12)$$

$$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!false \rightarrow \text{Skip})) \quad (13)$$

Rule 48. Loop Fragment Guard
$$\llbracket (\text{loop}(\text{min})[\text{condition}] \ x_1 \ \text{end}) : \text{LoopFragment} \rrbracket_G =$$

$\text{not}(\text{condition}) \&$	(1)
$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!0 \rightarrow$	(2)
$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!\text{false} \rightarrow \text{Skip})$	(3)
$\square$	(4)
$((\text{count} < \text{min}) \ \text{and} \ \text{condition}) \&$	(5)
$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!(\text{count} + 1) \rightarrow$	(6)
$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!\text{true} \rightarrow \text{Skip})$	(7)
$\square$	(12)
$((\text{count} == \text{min}) \ \text{and} \ \text{condition}) \&$	(13)
$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!0 \rightarrow$	(16)
$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!\text{false} \rightarrow \text{Skip})$	(17)

Rule 49. Loop Fragment Guard
$$\llbracket (\text{loop}[\text{condition}] \ x_1 \ \text{end}) : \text{LoopFragment} \rrbracket_G =$$

$\text{not}(\text{condition}) \&$	(1)
$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!0 \rightarrow$	(2)
$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!\text{false} \rightarrow \text{Skip})$	(3)
$\square$	(4)
$\text{condition} \&$	(5)
$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!(\text{count} + 1) \rightarrow$	(6)
$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!\text{true} \rightarrow \text{Skip})$	(7)

Rule 50. Loop Fragment Guard
$$\llbracket (\text{loop}(\text{min}) \ x_1 \ \text{end}) : \text{LoopFragment} \rrbracket_G =$$

$(\text{count} < \text{min}) \&$	(1)
$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!(\text{count} + 1) \rightarrow$	(2)
$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!\text{true} \rightarrow \text{Skip})$	(3)
$\square$	(4)
$(\text{count} == \text{min}) \&$	(5)
$(\text{setCount.ID_LOOP.id}(\text{loopFrag})!0 \rightarrow$	(6)
$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!\text{false} \rightarrow \text{Skip})$	(7)

Rule 51. Loop Fragment Guard
$$\llbracket (\text{loop} \ x_1 \ \text{end}) : \text{LoopFragment} \rrbracket_G =$$

$\text{guard.ID_LOOP.id}(\text{loopFrag}).1!\text{true} \rightarrow \text{Skip}$	(1)
---	-----

Rule 52. Alt Fragment

$$\llbracket (\text{alt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] x_1 \text{ else } \dots \text{ else } [g_{-n_0} \& g_{-n_1} \& \dots \& g_{-n_n}] x_n \text{ else } x_m \text{ end}) : \mathbf{AltFragment}, \mathbf{a} : \mathbf{Actor} \rrbracket_{AF} =$$

$$\text{alt.ID_ALT.id}(\text{altFrag}) \rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).1?id1 \quad (1)$$

$$\rightarrow \dots \rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).1?idn \quad (2)$$

$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF}) \sqcap \dots \sqcap idn \& (\llbracket a, x_n \rrbracket_{LIF})) \quad (3)$$

$$\sqcap \text{not}(id1 \text{ or } \dots \text{ or } idn) \& (\llbracket a, x_m \rrbracket_{LIF})) \quad (4)$$

Rule 53. Alt Fragment

$$\llbracket (\text{alt } x_1 \text{ else } \dots \text{ else } x_n \text{ end}) : \mathbf{AltFragment}, \mathbf{a} : \mathbf{Actor} \rrbracket_{AF} =$$

$$\text{alt.ID_ALT.id}(\text{altFrag}) \rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).1?id1 \quad (1)$$

$$\rightarrow \dots \rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).1?idn \quad (2)$$

$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF}) \sqcap \dots \sqcap idn \& (\llbracket a, x_n \rrbracket_{LIF}))$$

Rule 54. Alt Fragment

$$\llbracket (\text{alt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] x_1 \text{ else } x_2 \text{ end}) : \mathbf{AltFragment}, \mathbf{a} : \mathbf{Actor} \rrbracket_{AF} =$$

$$\text{alt.ID_ALT.id}(\text{altFrag}) \rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).1?id1 \quad (1)$$

$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF})) \quad (3)$$

$$\sqcap \text{not}(id1) \& (\llbracket a, x_2 \rrbracket_{LIF})) \quad (4)$$

Rule 55. Alt Fragment

$$\llbracket (\text{alt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] x_1 \text{ end}) : \mathbf{AltFragment}, \mathbf{a} : \mathbf{Actor} \rrbracket_{AF} =$$

$$\text{alt.ID_ALT.id}(\text{altFrag}) \rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).1?id1 \quad (1)$$

$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF})) \quad (3)$$

$$\sqcap \text{not}(id1) \& (\mathbf{Skip})) \quad (4)$$

Rule 56. Alt Fragment

$$\llbracket (\text{alt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] x_1 \text{ else } \dots \text{ else } [g_{-n_0} \& g_{-n_1} \& \dots \& g_{-n_n}] x_n \text{ end}) : \mathbf{AltFragment}, \mathbf{a} : \mathbf{Actor} \rrbracket_{AF} =$$

$$\text{alt.ID_ALT.id}(\text{altFrag}) \rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).1?id1 \quad (1)$$

$$\rightarrow \dots \rightarrow \text{guard.ID_ALT.id}(\text{altFrag}).1?idn \quad (2)$$

$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF}) \sqcap \dots \sqcap idn \& (\llbracket a, x_n \rrbracket_{LIF})) \quad (3)$$

$$\sqcap \text{not}(id1 \text{ or } \dots \text{ or } idn) \& (\mathbf{Skip})) \quad (4)$$

Rule 57. Opt Fragment
$$\llbracket (\text{opt } [g_{-1_0} \& g_{-1_1} \& \dots \& g_{-1_n}] \ x_1 \ \text{end}) : \mathbf{OptFragment}, \ a : \mathbf{Actor} \rrbracket_{OF} =$$

$$\text{opt.ID_OPT.id}(\text{optFrag}) \rightarrow \text{guard.ID_OPT.id}(\text{optFrag}).1?id1 \quad (1)$$

$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF}) \sqcap \text{not}(id1) \& (\mathbf{Skip})) \quad (2)$$

Rule 58. Loop Fragment
$$\llbracket (\text{loop } x_1 \ \text{end}) : \mathbf{LoopFragment}, \ a : \mathbf{Actor} \rrbracket_{LF} =$$

$$\text{let} \quad \text{Loop} = \quad (1)$$

$$\text{loop.ID_LOOP.id}(\text{loopFrag}) \rightarrow \text{guard.ID_LOOP.id}(\text{loopFrag}).1?id1 \quad (2)$$

$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF} \rightarrow \text{Loop}) \sqcap \text{not}(id1) \& (\mathbf{Skip})) \quad (3)$$

$$\text{within} \quad \text{Loop} \quad (4)$$

Rule 59. Loop Fragment
$$\llbracket (\text{loop}[\text{condition}] \ x_1 \ \text{end}) : \mathbf{LoopFragment}, \ a : \mathbf{Actor} \rrbracket_{LF} =$$

$$\text{let} \quad \text{Loop} = \quad (1)$$

$$\text{loop.ID_LOOP.id}(\text{loopFrag}) \rightarrow \text{guard.ID_LOOP.id}(\text{loopFrag}).1?id1 \quad (2)$$

$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF} \rightarrow \text{Loop}) \sqcap \text{not}(id1) \& (\mathbf{Skip})) \quad (3)$$

$$\text{within} \quad \text{Loop} \quad (4)$$

Rule 60. Loop Fragment
$$\llbracket (\text{loop}(\text{min}) \ x_1 \ \text{end}) : \mathbf{LoopFragment}, \ a : \mathbf{Actor} \rrbracket_{LF} =$$

$$\text{let} \quad \text{Loop} = \quad (1)$$

$$\text{loop.ID_LOOP.id}(\text{loopFrag}) \rightarrow \text{guard.ID_LOOP.id}(\text{loopFrag}).1?id1 \quad (2)$$

$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF} \rightarrow \text{Loop}) \sqcap \text{not}(id1) \& (\mathbf{Skip})) \quad (3)$$

$$\text{within} \quad \text{Loop} \quad (4)$$

Rule 61. Loop Fragment
$$\llbracket (\text{loop}(\text{min})[\text{condition}] \ x_1 \ \text{end}) : \mathbf{LoopFragment}, \mathbf{a} : \mathbf{Actor} \rrbracket_{LF} =$$

let

$$\text{Loop} = \quad (1)$$
$$\text{loop.ID_LOOP.id}(\text{loopFrag}) \rightarrow \text{guard.ID_LOOP.id}(\text{loopFrag}).1?id1 \quad (2)$$
$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF} \rightarrow \text{Loop}) \sqcap \text{not}(id1) \& (\mathbf{Skip})) \quad (3)$$

within

$$\text{Loop} \quad (4)$$

Rule 62. Loop Fragment
$$\llbracket (\text{loop}(\text{min}, \text{max}) \ x_1 \ \text{end}) : \mathbf{LoopFragment}, \mathbf{a} : \mathbf{Actor} \rrbracket_{LF} =$$

let

$$\text{Loop} = \quad (1)$$
$$\text{loop.ID_LOOP.id}(\text{loopFrag}) \rightarrow \text{guard.ID_LOOP.id}(\text{loopFrag}).1?id1 \quad (2)$$
$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF} \rightarrow \text{Loop}) \sqcap \text{not}(id1) \& (\mathbf{Skip})) \quad (3)$$

within

$$\text{Loop} \quad (4)$$

Rule 63. Loop Fragment
$$\llbracket (\text{loop}(\text{min}, \text{max})[\text{condition}] \ x_1 \ \text{end}) : \mathbf{LoopFragment}, \mathbf{a} : \mathbf{Actor} \rrbracket_{LF} =$$

let

$$\text{Loop} = \quad (1)$$
$$\text{loop.ID_LOOP.id}(\text{loopFrag}) \rightarrow \text{guard.ID_LOOP.id}(\text{loopFrag}).1?id1 \quad (2)$$
$$\rightarrow (id1 \& (\llbracket a, x_1 \rrbracket_{LIF} \rightarrow \text{Loop}) \sqcap \text{not}(id1) \& (\mathbf{Skip})) \quad (3)$$

within

$$\text{Loop} \quad (4)$$

Rule 64. Shared Variables List
$$\llbracket \mathbf{vars} : \text{seq } \mathbf{VariableList} \rrbracket_{SVARS} \hat{=}$$

$$\bigcup \{ \text{vs} : \text{ran } \mathbf{vars} \bullet \llbracket \text{vs.vars} \rrbracket_{SVAR} \} \quad (1)$$

Rule 65. Shared Variables
$$\llbracket \mathbf{vars} : \text{seq } \mathbf{Variable} \rrbracket_{SVAR} \hat{=}$$

$$\{ | \ \underline{v} : \mathbf{vars} \bullet \text{get_id}(\underline{v}), \text{set_id}(\underline{v}) \ | \} \quad (1)$$

Rule 66. Memory Variable List

$$\llbracket \text{varList} : \text{VariableList}, \text{varLists} : \text{seq VariableList} \rrbracket_{MEMV} \hat{=} \\ \llbracket \text{varList} \rrbracket_{MEM} \underline{(\text{if } \text{varLists} \neq \text{null} \text{ then } \llbracket \text{varLists} \rrbracket_{MEMV} _)} \quad (1)$$

Rule 67. Memory

$$\llbracket \text{varList} : \text{VariableList} \rrbracket_{MEM} \hat{=} \\ (\mid \mid \text{var} : \text{varList.vars} \bullet [\{ \mid \text{terminate} \mid \}] \llbracket \text{var} \rrbracket_{MEMVARS}) \quad (1)$$

Rule 68. Variable Memory

$$\llbracket \text{var} \rrbracket_{MEMVARS} \hat{=} \\ \text{let} \\ \quad \text{Memory}(\text{var}) = \quad (1) \\ \quad \quad \text{get_id!var} \rightarrow \text{Memory}(\text{var}) \quad (2) \\ \quad \quad \square \text{ set_id?x} \rightarrow \text{Memory}(\text{var}) \quad (3) \\ \quad \quad \square \text{ terminate} \rightarrow \text{Skip} \quad (4) \\ \text{within} \\ \quad \text{Memory}(\underline{\text{initial}(\text{var})}) \quad (5) \\ \text{where} \\ \quad \text{id} = \underline{\text{id}(\text{var})} \quad (6)$$

Rule 69. Deterministic Wait

$$\llbracket (\text{wait}(x) \text{ on } a) : \text{WaitOccurrence} \rrbracket_{(a)}^F = \\ \text{wait}(x) \quad (1)$$

Rule 70. Nondeterministic Wait

$$\llbracket (\text{wait}([v, y]) \text{ on } a) : \text{WaitOccurrence} \rrbracket_{(a)}^F = \\ (\sqcap x : \{v..y\} \bullet \text{wait}(x)) \quad (1)$$

Rule 71. Deadline

$$\llbracket (\text{deadline}(x) \text{ b}) : \text{DeadlineFragment} \rrbracket_{(a)}^F = \\ \text{Deadline}(\llbracket a, b \rrbracket_{LIF}, x) \quad (1)$$

Rule 72. Destroy

$$\begin{array}{l} \llbracket (\text{destroy on } a) : \mathbf{DestroyOccurrence} \rrbracket_{(a)}^F = \\ \text{terminate} \rightarrow \mathbf{Skip} \end{array} \quad (1)$$

Rule 73. Ignore

$$\begin{array}{l} \llbracket (\text{ignore}(m_1, m_2, \dots, m_n) \ b) : \mathbf{IgnoreFragment} \rrbracket_{(a)}^F = \\ (b) \setminus \{m_1, m_2, \dots, m_n\} \end{array} \quad (1)$$

Rule 74. Consider

$$\begin{array}{l} \llbracket (\text{consider}(m_1, m_2, \dots, m_n) \ b) : \mathbf{IgnoreFragment} \rrbracket_{(a)}^F = \\ (b) \setminus \{ALL \setminus \{m_1, m_2, \dots, m_n\}\} \end{array} \quad (1)$$

where

$$ALL = \text{allFrag}(\text{b}) \quad (2)$$

Rule 75. Message - Source Actor

$$\begin{array}{l} \llbracket (a \rightarrow a_1 : \text{mID}) : \mathbf{InteractionFragment} \rrbracket_{(a)}^F = \\ \text{mID} \rightarrow \mathbf{Skip} \end{array} \quad (1)$$

Rule 76. Message - Target Actor

$$\begin{array}{l} \llbracket (a_1 \rightarrow a : \text{mID}) : \mathbf{InteractionFragment} \rrbracket_{(a)}^F = \\ \text{mID} \rightarrow \mathbf{Skip} \end{array} \quad (1)$$

Rule 77. Message With Output Constant - Source Actor

$$\begin{array}{l} \llbracket (a \rightarrow a_1 : \text{mID!var}) : \mathbf{InteractionFragment} \rrbracket_{(a)}^F = \\ \text{mID!var} \rightarrow \mathbf{Skip} \end{array} \quad (1)$$

Rule 78. Message With Output Constant - Target Actor

$$\llbracket (a_1 \rightarrow a : \text{mID!var}) : \text{InteractionFragment} \rrbracket_{(a)}^F =$$

$$\text{mID?var} \rightarrow \text{set_var!var} \rightarrow \text{Skip} \quad (1)$$

Rule 79. Message With Output Literal - Source Actor

$$\llbracket (a \rightarrow a_1 : \text{mID!val}) : \text{InteractionFragment} \rrbracket_{(a)}^F =$$

$$\text{mID!val} \rightarrow \text{Skip} \quad (1)$$

Rule 80. Message With Output Literal - Target Actor

$$\llbracket (a_1 \rightarrow a : \text{mID!val}) : \text{InteractionFragment} \rrbracket_{(a)}^F =$$

$$\text{mID?var} \rightarrow \text{set_var!var} \rightarrow \text{Skip} \quad (1)$$

where

$$\text{var} = \text{var}(\text{val}) \quad (2)$$

Rule 81. Message With Input - Source Actor

$$\llbracket (a \rightarrow a_1 : \text{mID?var}) : \text{InteractionFragment} \rrbracket_{(a)}^F =$$

$$\text{mID?var} \rightarrow \text{set_var!var} \rightarrow \text{Skip} \quad (1)$$

Rule 82. Message With Input - Target Actor

$$\llbracket (a_1 \rightarrow a : \text{mID?var}) : \text{InteractionFragment} \rrbracket_{(a)}^F =$$

$$\text{mID!var} \rightarrow \text{Skip} \quad (1)$$

Rule 83. Self Message Actor

$$\llbracket (a \rightarrow a : \text{mID?var}) : \text{InteractionFragment} \rrbracket_{(a)}^F =$$

$$\text{mID?var} \rightarrow \text{set_var!var} \rightarrow \text{Skip} \quad (1)$$
